

Word embeddings

[attachment:c9a69743-6c22-475e-8fc6-7d340f3a7bd4:04_Word_embeddings.pdf](#)

similar words should have similar vectors and properties

negative sampling in **Word2vec**

Word Embeddings: Slide-Paired Notes

Source slides: [/Users/student/Downloads/04_Word_embeddings.pdf](#)

Supporting source: [/Users/student/.codex/output/transcribe/20260311-130709/transcript.md](#)

These notes are meant to be read while going through the presentation. The goal is to capture what the lecturer adds beyond the slide bullets, especially explanation, intuition, and exam-relevant contrasts.

Final short study summary

- Bag-of-words fails because it is sparse, ignores word order, and cannot represent semantic similarity.
- Word embeddings solve this by mapping words into dense continuous vector spaces.
- Similar meaning is learned from similar context.
- Context definition matters: small windows are more syntactic, larger windows more semantic.
- Neural language models learn embeddings through word prediction.
- Word2vec makes this more efficient, mainly through CBOW, skip-gram, and tricks like negative sampling.

- CBOW predicts the target from context; skip-gram predicts context from the target.
- GloVe learns from global co-occurrence statistics, especially probability ratios.
- FastText uses character n-grams, which helps with morphology and unseen words.
- Static embeddings give one vector per word; contextual embeddings give vectors that depend on the sentence.

Quick examples to remember

- **A is better than B** VS **B is better than A**
Bag-of-words treats them as the same, embeddings aim to distinguish meaning through context.
- **ice** VS **steam**
The useful distinction comes from context words like **solid** or **gas**, not generic words like **water**.
- **bank**
Static embedding mixes financial-bank and river-bank meanings; contextual embedding separates them based on the sentence.

Slide 1 - Word embeddings

This is the start of the topic: the lecture moves away from sparse text representations and toward dense vector representations of words. The main question for the rest of the deck is how to represent meaning so that similar words are close to each other and can be used by machine learning models more effectively.

Short takeaway

Word embeddings = dense numeric vectors that try to capture word meaning and relationships.

Slide 2 - Limitations of the bag-of-words model

The lecturer emphasizes why bag-of-words is not enough for meaning. It creates very large sparse vectors, ignores word order, and treats words as independent features. As the lecturer points out, sentences like "A is better

than B" and "B is better than A" become identical if we only track which words appear. The model also cannot capture that some words are semantically related.

Short takeaway

Bag-of-words is simple, but it loses order, structure, and semantic similarity.

Slide 3 - Sparse vectors example

As we can see on this slide, even very short texts become huge sparse vectors in a vocabulary-sized space. The lecturer uses this kind of example to show two core problems:

- the number of dimensions can reach tens or hundreds of thousands
- different text vectors are often almost equally far from each other, so similarity becomes uninformative

Short takeaway

Sparse high-dimensional vectors are inefficient and do not represent meaning well.

Slide 4 - Eliminating some problems

The lecturer frames this slide as a transition: before embeddings, there are partial fixes. Feature selection can remove useless words, dimensionality reduction can compress representations, and adding context can preserve more information. But these approaches still do not fully solve the core problem, which is representing meaning in a useful way.

Short takeaway

Embeddings are presented as a stronger solution than just filtering or compressing bag-of-words features.

Slide 5 - Storing some context

The lecturer explains that one possible improvement is to enrich word representation with extra information such as POS tag, previous word, or next word. This helps preserve context, but it also explodes dimensionality even

more. So context is useful, but we need a smarter way to include it than adding more sparse columns.

Short takeaway

Context matters, but naive context features make the representation even bigger.

Slide 6 - Word embeddings

This is the conceptual definition. Instead of representing a word by one independent dimension, we assign it a lower-dimensional vector in a continuous space. The lecturer stresses that similar words should share properties, so their vectors should be similar too. Unlike bag-of-words, embeddings make similarity part of the representation itself.

Short takeaway

One word is no longer one isolated feature. It becomes a point in a semantic vector space.

Slide 7 - Visual example

This slide likely shows an illustrative embedding plot. The lecturer's message here is that embeddings create a geometric space where distance and direction become meaningful.

Short takeaway

Embeddings can be visualized as a space where meaning has geometry.

Slide 8 - Properties of word vectors

The lecturer explains two famous properties:

- similar words tend to have similar vectors
- relationships between word pairs can appear as similar vector offsets

This is where the idea of analogy-like reasoning comes from. The point is not that embeddings "understand" language perfectly, but that these vector operations reveal useful regularities.

Short takeaway

Embeddings can encode both similarity and certain semantic relations.

Slide 9 - Properties of word vectors (visual)

As we can see on this slide, the visual probably demonstrates that related pairs of words have similar directional offsets. In the lecture, examples like male-female or country-related relations are used to show that vector arithmetic can capture structured relationships.

Short takeaway

Vector directions can carry relational information, not just closeness.

Slide 10 - Learning word embeddings

The lecturer says we cannot manually design hundreds of dimensions for hundreds of thousands of words. We need an automatic learning process. The key idea introduced here is the distributional hypothesis: words used in similar contexts tend to have similar meanings. The Firth quote gives the intuition behind most classical embedding methods.

Short takeaway

"A word is characterized by the company it keeps" is the central idea behind classical embeddings.

Slide 11 - The context

This slide is important for understanding what "context" means. The lecturer explains that context can be:

- words in a small local window
- positions relative to the target word
- syntactic dependencies

The lecturer also points out a useful distinction:

- small windows capture more syntactic or functional similarity
- larger windows capture more semantic similarity

This is exam-relevant because it explains why different setups learn different kinds of information.

Short takeaway

The definition and size of context strongly affect what the embeddings learn.

Slide 12 - Computing word embeddings: supervised methods

Here the lecturer contrasts supervised and unsupervised approaches. In supervised learning, embeddings are optimized for a specific annotated task, such as POS tagging. Because of that, they often capture information useful for that task and can sometimes transfer to related tasks.

Short takeaway

Supervised embeddings are task-oriented and depend on labeled data.

Slide 13 - Computing word embeddings: unsupervised methods

The lecturer notes that in practice, unsupervised or self-supervised methods are more common for learning general-purpose embeddings. Instead of needing labels, they generate training signal from raw text itself, usually by predicting a word from context or vice versa. This allows training on very large corpora.

Short takeaway

Self-supervision learns embeddings from plain text, which makes it scalable and broadly useful.

Slide 14 - Neural language model

This slide introduces the language-modeling objective: predict the next word from previous words. The lecturer presents this as one of the early neural ways to learn word representations automatically. The model learns parameters that make the predicted probabilities match the actual text.

Short takeaway

A neural language model learns embeddings by trying to predict words in sequences.

Slide 15 - Neural language model architecture

As we can see on this slide, the model uses:

- one-hot input vectors for words
- an embedding matrix that converts them into dense vectors
- concatenation of context embeddings
- hidden layers
- a softmax output over the whole vocabulary

The lecturer explains that the embedding matrix is where the word vectors live. The neural network is trained for prediction, but the learned embedding matrix is what we keep afterward.

Short takeaway

The embedding matrix is a learned lookup table from one-hot words to dense vectors.

Slide 16 - Learning process in the neural language model

The lecturer connects this to standard neural network training: forward pass, loss computation, backpropagation, and parameter update. A key point is that the embedding matrix is updated during backpropagation too, so the word vectors are learned jointly with the rest of the model.

Short takeaway

Embeddings are learned as model parameters, not added afterward.

Slide 17 - Neural language model diagram

This is probably a structural visualization of the previous two slides. The lecturer's point is to help students see where embeddings enter the network and how they support prediction.

Short takeaway

The embedding layer is part of a predictive network, not a separate preprocessing step.

Slide 18 - Word2vec overview

The lecturer introduces word2vec as a more computationally efficient family of models than earlier neural language models. The key contrast is that word2vec simplifies the architecture and often replaces expensive full softmax computation with cheaper alternatives such as hierarchical softmax or negative sampling.

Short takeaway

Word2vec keeps the predictive idea but makes training much cheaper.

Slide 19 - Word2vec CBOW

CBOW predicts the center word from the surrounding context. The lecturer explains that context word embeddings are averaged first, then used to score candidate target words. This averaging makes CBOW fast and simple, but it also loses word order and gives all context words equal influence.

Short takeaway

CBOW = context words in, target word out.

Slide 20 - Word2vec skip-gram

Skip-gram does the reverse: it starts from one target word and predicts surrounding context words. The lecturer points out that this creates a training signal for each context word separately. That usually gives stronger updates and can lead to richer semantic representations, especially for less frequent words.

Short takeaway

Skip-gram = target word in, context words out.

Slide 21 - Two embedding matrices

The lecturer explains why word2vec often has two vectors per word during training:

- one when the word appears as the input or target
- one when the word appears in context or output

After training, one vector set can be used, or both can be averaged. The lecturer notes that averaging often works slightly better.

Short takeaway

Words play different roles during training, so two embedding matrices are learned.

Slide 22 - Word2vec diagram

This slide likely visualizes one of the word2vec architectures. The lecture uses it to make the data flow more concrete, especially the role of the embedding matrix and output scoring.

Short takeaway

Word2vec is easier to train than the earlier neural language model because the architecture is simpler.

Slide 23 - Negative sampling

As we can see on this slide, the lecturer explains negative sampling as a practical replacement for full-vocabulary softmax. Instead of comparing the correct word to every word in the dictionary, the model compares it to only a few randomly sampled incorrect words. The goal is:

- score the correct word-context pair high
- score sampled incorrect pairs low

This is one of the main reasons word2vec became efficient enough to train at scale.

Short takeaway

Negative sampling turns a huge prediction problem into a much smaller discrimination problem.

Slide 24 - Why skip-gram is often strong

The lecturer highlights an important comparison:

- CBOW is faster
- skip-gram handles rare words better and often learns richer semantics

The explanation given in the lecture is that CBOW averages context vectors, so each word gets a diluted update, while skip-gram treats word-context relations more separately.

Short takeaway

Skip-gram is slower than CBOW but often better for rare words and semantic quality.

Slide 25 - GloVe overview

The lecturer now shifts from predictive models to count-based global methods. GloVe uses co-occurrence statistics from the entire corpus, not just local prediction in individual windows. This means it captures broader global information about how words appear together.

Short takeaway

GloVe is based on corpus-wide co-occurrence statistics.

Slide 26 - Why ratios matter in GloVe

The lecturer emphasizes that raw co-occurrence counts or single probabilities do not capture meaning very well on their own. Ratios such as $P(k|i) / P(k|j)$ tell us whether a context word k is more associated with word i or with word j . That makes semantic differences easier to identify.

Short takeaway

GloVe uses probability ratios because they reveal discriminative semantic information.

Slide 27 - Ice vs steam example

As we can see on this slide, the ice/steam example shows why ratios are better than raw counts. Words like **solid** strongly prefer **ice**, words like **gas** strongly prefer **steam**, while words like **water** are common to both and therefore less informative. The lecturer uses this example to show how discriminative context works in practice.

Short takeaway

Informative context words help distinguish meanings; generic context words do not.

Slide 28 - GloVe objective

This slide gives the formal objective. The lecturer's core message is simpler than the formula: GloVe learns vectors so that dot products and biases approximate log co-occurrence information, while a weighting function reduces the influence of extremely frequent co-occurrences.

Short takeaway

GloVe fits vectors to global co-occurrence structure using a weighted objective.

Slide 29 - Final GloVe vectors

Like word2vec, GloVe learns two vector sets, one for center words and one for context words. The lecturer notes that the final embedding is usually formed by summing them.

Short takeaway

GloVe also ends with two learned vector roles that are usually combined.

Slide 30 - FastText motivation

The lecturer explains that standard embeddings treat a word as a whole unit and ignore its internal structure. This is a problem for morphologically rich languages and rare word forms. If a word or inflected form appears rarely, the model may not learn a good vector for it.

Short takeaway

Classical word embeddings struggle with morphology and rare or unseen word forms.

Slide 31 - FastText idea

FastText extends word2vec by representing a word as a bag of character n-grams. The lecturer explains that:

- each n-gram has its own vector

- the word vector is the sum of its n-gram vectors
- this helps share information across related word forms

That is why FastText can create embeddings even for words not seen during training.

Short takeaway

FastText uses subword pieces, which helps with morphology and out-of-vocabulary words.

Slide 32 - Word vector training

The lecturer briefly notes that embeddings are usually initialized randomly, like other model parameters. The important idea is that meaning is not built in from the start; it emerges through optimization.

Short takeaway

Word vectors start random and become meaningful through training.

Slide 33 - Using word vectors

This slide moves from learning embeddings to applying them. The lecturer explains that if we want to represent sentences, paragraphs, or documents, we need to aggregate word vectors into one fixed-length representation, often by sum or average. These aggregated vectors can then be used in classifiers, clustering, or as inputs to larger neural architectures.

Short takeaway

Word vectors are often building blocks for larger text representations.

Slide 34 - Static embeddings

This is one of the most important conceptual distinctions near the end of the lecture. The lecturer explains that all methods discussed so far are static:

- one word gets one fixed vector
- this is a problem for polysemous words like **bank**

Contextual models such as BERT or GPT instead compute a representation that depends on the sentence. The lecturer stresses that static embeddings mix

multiple meanings together, while contextual embeddings adapt to the actual usage.

Short takeaway

Static embeddings ignore sense differences across contexts; contextual embeddings do not.

Slide 35 - Using static embeddings

The lecturer closes by explaining why static embeddings are still useful:

- pretrained embeddings are easy to obtain
- they are lightweight in memory and computation
- they are good for baselines and simpler tasks
- they can speed up training when used to initialize models

So even though contextual models are stronger, static embeddings still have practical value.

Short takeaway

Static embeddings are older, but still useful when simplicity, speed, or limited resources matter.